

# Step-by-Step Guide: Migrating Kedro-AzureML Project to Compute Cluster

This guide outlines the process of transitioning your `house-price` Kedro project from an Azure ML Compute Instance to a Compute Cluster, specifically addressing the challenge of restricted root access for Docker image pulls.

## 1. Understanding the Root Access Limitation

Currently, your data scientists face job submission errors because their individual **Compute Instances** lack root access. This prevents the instances from pulling necessary Docker images from the Microsoft Container Registry (MCR) or building custom environments required for their pipelines. This security measure, while important for private organizations, inadvertently blocks critical development workflows.

## 2. Why Azure ML Compute Clusters Solve This

**Azure ML Compute Clusters** are designed for scalable, shared, and managed execution of machine learning workloads. Unlike Compute Instances, which are personal workstations, clusters are pools of virtual machines that can automatically scale up and down based on demand. The key advantages for your scenario are:

- **Managed Identity for Permissions:** Compute Clusters can be assigned a Managed Identity, which Azure ML uses to securely authenticate and authorize actions, such as pulling Docker images from MCR or your Azure Container Registry (ACR), without granting root access to individual users.
- **Scalability and Cost-Efficiency:** Clusters automatically provision resources as needed and scale to zero nodes when idle, significantly reducing costs compared to always-on Compute Instances.
- **Job Isolation:** Each job submitted to a cluster runs in its own isolated environment, preventing conflicts between concurrent runs from different data scientists.

## 3. Step-by-Step Migration Implementation

This section details the necessary steps to migrate your Kedro project.

### Step 3.1: Create an Azure ML Compute Cluster

First, you need to set up a Compute Cluster in your Azure ML Workspace. This can be done via the Azure Portal or Azure CLI.

## Option A: Azure Portal

1. Navigate to [Azure ML Studio](#) and select your workspace.
2. In the left navigation pane, select **Compute**.
3. Go to the **Compute clusters** tab and click **+ New**.
4. Configure the cluster with the following settings:
  - **Virtual machine priority:** `Dedicated` (recommended for production workloads).
  - **Virtual machine size:** Choose an appropriate VM size (e.g., `Standard_DS11_v2` or `Standard_E2s_v3`) based on your workload requirements.
  - **Cluster name:** Provide a unique name (e.g., `shared-training-cluster`).
  - **Minimum number of nodes:** `0` (for cost efficiency).
  - **Maximum number of nodes:** `20` (to accommodate all 20 data scientists running concurrently).
  - **Idle seconds before scale down:** `120` (or a suitable value to quickly scale down idle nodes).
5. Click **Create**.

## Option B: Azure CLI

Ensure you have the Azure ML CLI extension installed ( `az extension add --name ml` ).

Bash

```
az ml compute create \  
  --name shared-training-cluster \  
  --type amlcompute \  
  --size Standard_DS11_v2 \  
  --min-instances 0 \  
  --max-instances 20 \  
  --idle-time-before-scale-down 120
```

## Step 3.2: Create a Custom Azure ML Environment

Since your project relies on specific Python packages ( `requirements.txt` ), you need to create a custom Azure ML Environment. This environment will encapsulate your project's dependencies and will be built as a Docker image by Azure ML. The Compute Cluster will then use this image to run your jobs.

1. **Ensure `requirements.txt` is up-to-date:** Your project's `requirements.txt` file ( `/home/ubuntu/learn-with-me-kedro-azml/house-price/requirements.txt` ) lists all necessary Python packages. This file is crucial for defining your environment.

## 2. Create the Environment via Python SDK (Recommended for Automation):

You can define and register an environment programmatically. Create a Python script (e.g., `create_environment.py`) in your project root:

Python

```
# create_environment.py
from azure.ai.ml import MLClient
from azure.ai.ml.entities import Environment
from azure.identity import DefaultAzureCredential

# Replace with your Azure subscription, resource group, and workspace name
subscription_id = "<YOUR_SUBSCRIPTION_ID>"
resource_group = "<YOUR_RESOURCE_GROUP>"
workspace_name = "<YOUR_WORKSPACE_NAME>"

ml_client = MLClient(
    DefaultAzureCredential(), subscription_id, resource_group, workspace_name
)

# Define the environment from requirements.txt
custom_env_name = "kedro-house-price-env"
env_docker_conda = Environment(
    name=custom_env_name,
    image="mcr.microsoft.com/azureml/openmpi4.1.0-ubuntu20.04:latest", # A c
    conda_file="requirements.txt", # Path to your requirements.txt
    description="Custom environment for Kedro house price project"
)

ml_client.environments.create_or_update(env_docker_conda)
print(f"Environment '{custom_env_name}' created/updated successfully.")
```

### Explanation:

Run this script from your Compute Instance's terminal (or locally if configured with Azure CLI):

Bash

```
python create_environment.py
```

- `image` : We use a base image from MCR. The Compute Cluster, through its Managed Identity, will have the necessary permissions to pull this base image.
- `conda_file` : This points to your `requirements.txt`. Azure ML will automatically install these packages on top of the base image when building the custom Docker image

for your environment.

### Step 3.3: Update Kedro Project for Azure ML Job Submission

Instead of running `kedro run` directly on the Compute Instance, you will now submit your Kedro pipeline as an Azure ML Job to the Compute Cluster. This requires modifying your Kedro project to integrate with the Azure ML SDK.

1. **Install Azure ML SDK:** Ensure your local development environment (e.g., your Compute Instance's kernel) has the Azure ML SDK installed.

Bash

```
pip install azure-ai-ml azure-identity
```

2. **Create an Azure ML Job Submission Script:**

In your Kedro project root, create a Python script (e.g., `submit_to_azureml.py`) that defines and submits your Kedro pipeline as an Azure ML command job.

Python

```
# submit_to_azureml.py
from azure.ai.ml import MLClient, command
from azure.ai.ml.entities import Environment
from azure.identity import DefaultAzureCredential
import os

# Replace with your Azure subscription, resource group, and workspace name
subscription_id = "<YOUR_SUBSCRIPTION_ID>"
resource_group = "<YOUR_RESOURCE_GROUP>"
workspace_name = "<YOUR_WORKSPACE_NAME>"

ml_client = MLClient(
    DefaultAzureCredential(), subscription_id, resource_group, workspace_name
)

# Get the custom environment created in Step 3.2
custom_env = ml_client.environments.get(name="kedro-house-price-env", version="1.0.0")

# Define the command job
job = command(
    name="kedro-house-price-pipeline",
    display_name="Kedro House Price Prediction Pipeline",
    command="python -m kedro run", # The command to execute your Kedro pipeline
    environment=custom_env, # Use the custom environment
    compute="shared-training-cluster", # Target the Compute Cluster
    code=".", # Path to your Kedro project root (current directory)
```

```

    experiment_name="house-price-prediction-experiment",
    description="Runs the Kedro house price prediction pipeline on Azure ML
)

# Submit the job
returned_job = ml_client.jobs.create_or_update(job)
print(f"Job submitted. View at: {returned_job.studio_url}")

```

### Explanation:

- `command="python -m kedro run"` : This is the standard command to execute your Kedro pipeline.
- `environment=custom_env` : This links your job to the custom environment (Docker image) that contains all your project dependencies.
- `compute="shared-training-cluster"` : This explicitly tells Azure ML to run the job on the Compute Cluster you created.
- `code="."` : This specifies that the entire current directory (your Kedro project) should be uploaded to Azure ML for the job run.

## Step 3.4: Submit Your Kedro Pipeline as an Azure ML Job

From your Compute Instance's terminal (or locally if configured), run the submission script:

Bash

```
python submit_to_azureml.py
```

This will submit your Kedro pipeline to the `shared-training-cluster`. You can monitor the job's progress in the Azure ML Studio under the **Jobs** section.

## 4. Day-to-Day Workflow Changes for Data Scientists

| Aspect                         | Current (Compute Instance)                                         | Proposed (Compute Cluster)                                     |
|--------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------|
| <b>Interactive Development</b> | Direct execution in notebook cells.                                | Still possible on Compute Instance.                            |
| <b>Pipeline Execution</b>      | <code>kedro run</code> in terminal on instance.                    | <code>python submit_to_azureml.py</code> to cluster.           |
| <b>Environment Management</b>  | Manual <code>pip install</code> on instance; prone to root issues. | Managed by Azure ML Environment; automated Docker image build. |

|                 |                                            |                                           |
|-----------------|--------------------------------------------|-------------------------------------------|
| Resource Usage  | Instance runs continuously, accruing cost. | Cluster scales on demand; cost-efficient. |
| Troubleshooting | Debugging on instance.                     | Debugging via Azure ML Job logs.          |

**Impact on Notebooks:** Your existing notebooks will remain in the `~/cloudfiles/code/Users/<your-user>/house-price/notebooks` directory. For interactive exploration and development, you can still use your Compute Instance. However, for full pipeline runs, you will use the `submit_to_azureml.py` script to leverage the Compute Cluster.

## 5. Proof of Concept (POC) Recommendation

To ensure a smooth transition, I recommend a focused POC:

1. **Objective:** Verify that the Compute Cluster can successfully pull Docker images (base MCR image and custom environment image) and execute the Kedro pipeline without root access issues.
2. **Participants:** Involve 1-2 data scientists from your team.
3. **Steps:**
  - Follow Steps 3.1 to 3.4 to create the cluster, environment, and submit the `house-price` pipeline.
  - Monitor the job in Azure ML Studio to confirm successful completion and review logs for any environment-related errors.
  - Verify that the output artifacts (e.g., `metrics.json`, trained models) are generated as expected.
4. **Timeline:** This POC should take approximately **4 hours** to complete.

## 6. Estimated Timeline for Full Transition

- **Cluster and Environment Setup (Admin):** 2 hours
- **POC with 1-2 Data Scientists:** 4 hours
- **Team Rollout & Training:** 1-2 days (to guide all 20 data scientists in updating their submission scripts and understanding the new workflow).

**Total Estimated Time: Approximately 2 days.**

## References

- [1] Azure Machine Learning documentation on Environments:
- [2] Azure Machine Learning documentation on Compute Clusters:
- [3] Azure Machine Learning documentation on submitting jobs: